



ps Command in Linux

Introduction

Imagine you are working on a **Linux system** and need to:

- Check which processes are running.
- Find the Process ID (PID) of a specific application.
- Monitor system performance and troubleshoot issues.

How do you list and analyze running processes?

This is where the **ps (process status) command** helps!

Section 1: Learn

What is **ps**?

The **ps** command in Linux displays information about currently running processes. It helps:

1. Monitor active processes and their resource usage.
2. Find and manage specific processes by name or ID.
3. Troubleshoot system slowdowns and unresponsive programs.

Why Do We Need **ps**?

Without ps	With ps
No way to see running processes	Lists active processes instantly
Hard to find unresponsive programs	Identifies slow or stuck processes



Without ps	With ps
No way to get process IDs	Displays PIDs for process management
System performance issues go unnoticed	Helps monitor and control resource usage

How Does **ps** Work?

1. It captures a snapshot of current processes.
2. It displays process details such as PID, CPU usage, and memory usage.
3. You can filter or format the output for better readability.

An Interesting Anecdote

In early computing, **processes were managed manually** without a tracking tool. The **ps** command was introduced in Unix to **help users see and control running tasks**, making system administration easier!

Section 2: Practice

Step-by-Step Guide to Using **ps**

1. Viewing All Running Processes (**-e**)

```
# Show all processes running on the system  
ps -e
```

What happens here?

- Lists all active processes with their Process ID (PID).
-



2. Displaying Detailed Process Information (-ef)

```
# Show extended details of running processes
```

```
ps -ef
```

Column	Meaning
UID	User who started the process
PID	Process ID
PPID	Parent Process ID
C	CPU usage percentage
STIME	Process start time
TTY	Terminal associated with the process
CMD	Command that started the process

Why is this useful?

- Provides **detailed process information**, including the user running the process.

3. Finding a Process by Name (grep)

```
# Find all processes related to Firefox
```

```
ps -ef | grep firefox
```

How does this help?

- Quickly **locates specific processes** running on the system.



4. Displaying Processes in a Tree Format (**--forest**)

```
# Show parent-child relationship of processes
```

```
ps -ef --forest
```

What does this do?

- Displays **how processes are linked** (useful for debugging).

5. Viewing Only Processes of a Specific User (**-u**)

```
# Show processes started by a specific user (e.g., "john")
```

```
ps -u john
```

Why is this important?

- Helps **monitor user activity** on a multi-user system.

6. Listing Processes Using CPU & Memory (**aux**)

```
# Show CPU and memory usage per process
```

```
ps aux
```

Column	Meaning
%CPU	CPU usage percentage
U	
%MEM	Memory usage percentage
M	
VSZ	Virtual memory size



Column	Meaning
RSS	Resident Set Size (RAM usage)

Why is this useful?

- Identifies **high-resource consuming processes**.
-

7. Killing a Process Using **ps** and **kill**

```
# Find the PID of the process
```

```
ps -ef | grep my_program
```

```
# Kill the process by PID (e.g., 1234)
```

```
kill 1234
```

Why is this needed?

- Stops **unresponsive or unnecessary processes**.
-

Real-World Example

A **system administrator** notices a **slow server**. They need to:

1. **Check which processes are consuming high CPU.**
2. **Find the process ID (PID) of the resource-heavy application.**
3. **Terminate the problematic process safely.**

What did they do?

1. Used **ps aux --sort=-%CPU** to **find the highest CPU-consuming process**.



2. Ran `ps -ef | grep java` to get the process ID of a Java application.
3. Executed `kill 5678` to stop the unresponsive process.

Result? The server performance improved immediately!

Section 3: Know More

Frequently Asked Questions

1. How do I see only my own processes?

```
ps -u $USER
```

2. What is the difference between `ps` and `top`?

- `ps` shows a snapshot of current processes.
- `top` provides real-time monitoring.

3. How do I find the parent process of a running task?

```
ps -ef --forest | grep process_name
```

4. How do I find processes using a specific port?

```
sudo netstat -tulpn | grep :8080
```

5. Where can I practice process management?

- Use a Linux Virtual Machine.
 - Try monitoring background tasks on cloud servers.
-

Conclusion

The `ps` command helps monitor, manage, and troubleshoot running processes, making Linux systems more efficient.



Start by **listing active processes, filtering by user, and finding resource-heavy applications**, and soon you'll be **managing Linux tasks like an expert!**